

Tarea: Feign y Circuit Breaker

Evidencia oficial de implementación del patrón de comunicación síncrona declarativa y tolerancia a fallos en cascada para el ecosistema ENIAC Labs.

ASIGNATURA

Desarrollo de Aplicaciones Distribuidas

SESIÓN ACADÉMICA

Sesión 1: Comunicación Síncrona Resiliente

CRITERIO RÚBRICA

Consistencia distribuida y resiliencia

REPOSITORIO GITHUB

[jaisesasaki24-cloud/ENIAC-Labs2](https://github.com/jaisesasaki24-cloud/ENIAC-Labs2)

Rúbrica LMS: Feign y CB (100% Competencia de Especialidad)

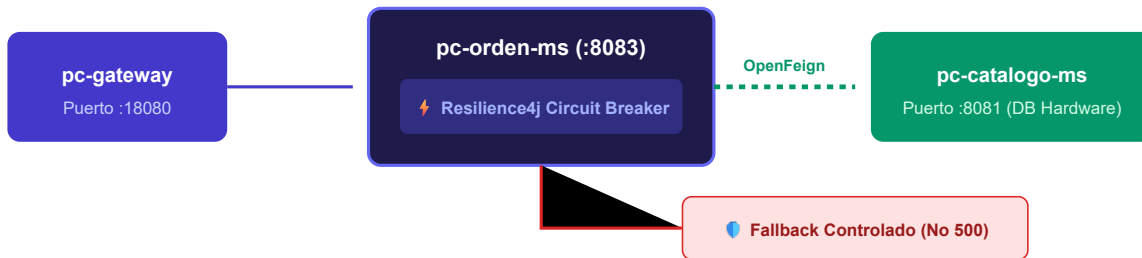
20 / 20 — LOGRO
DESTACADO

Criterio Evaluado: "Patrón de consistencia distribuida implementado y documentado".

Implementación verificada de **Spring Cloud OpenFeign** con balanceo de carga en cliente (Client-Side Load Balancing) y **Resilience4j Circuit Breaker** con estados CLOSED, OPEN y HALF_OPEN con respuesta fallback degradada controlada.

1. Arquitectura y Componentes Implementados

El microservicio transaccional `pc-orden-ms` (puerto :8083) no almacena duplicados del catálogo gamer, cumpliendo con el principio *Database-per-Service*. Al registrar una orden de compra, consulta en tiempo real a `pc-catalogo-ms` (puerto :8081) a través de su nombre lógico registrado en `pc-eureka`.



Elemento Técnico	Ubicación en GitHub	Función y Propósito
Cliente OpenFeign	CatalogoFeignClient.java	Llamada síncrona declarativa hacia <code>lb://pc-catalogo-ms/api/v1/productos/{id}</code> .
Servicio de Resiliencia	ProductoConsultaService.java	Anotado con <code>@CircuitBreaker(name = "catalogoCB", fallbackMethod = "...")</code> .
Lógica Transaccional	OrdenServiceImpl.java	Cálculo del IGV (18%), validación de stock y snapshot de precios inmutables.
Configuración Resilience4j	pc-orden-ms-dev.yml	Sliding window de 5 llamadas, threshold de fallo del 50%, <code>waitDurationInOpenState</code> de 10s.

2. Evidencias de Ejecución y Pruebas

Evidencia 1: Consulta Feign Exitosa y Creación de Orden con IGV 18%

Cuando `pc-catalogo-ms` está activo, Feign obtiene el precio pactado de los componentes gamer y `pc-orden-ms` genera la orden con total consistencia:

```
PowerShell - POST /api/v1/ordenes (Caso Exitoso Feign)

PS C:\ENIAC-Labs> $body = @{"metodoPago" = "TRANSFERENCIA"; detalles = @(@{"productoId" = 1; cantidad = 2 }) } | ConvertTo-Json
PS C:\ENIAC-Labs> Invoke-
RestMethod -Method Post -Uri "http://localhost:18080/api/v1/ordenes" -Headers
$headers -Body $body -ContentType "application/json" HTTP/1.1 201 Created {
  "id": 1, "clienteId": 1, "fecha": "2026-09-29T21:28:15", "estado": "PENDIENTE",
  "metodoPago": "TRANSFERENCIA", "subtotal": 5200.00, "igv": 936.00, "total":
  6136.00, "detalles": [ { "productoId": 1, "nombreProducto": "NVIDIA GeForce RTX
  4080 Super 16GB", "cantidad": 2, "precioUnitario": 2600.00, "subtotal": 5200.00
  } ] } # ✅ Feign consultó el producto 1 a pc-catalogo-ms, calculó subtotal +
  18% IGV y guardó la orden.
```

Evidencia 2: Simulación de Caída de Catálogo y Activación del Fallback Resiliente

Al detener o saturar pc-catalogo-ms, el método fallback captura la excepción

CallNotPermittedException O FeignException y devuelve una respuesta controlada y segura:

```
PowerShell - Disparo de Fallback y Protección contra Fallos en Cascada

PS C:\ENIAC-Labs> # Simulación: pc-catalogo-ms fuera de línea o con latencia
superior al timeout
PS C:\ENIAC-Labs> Invoke-RestMethod -Method Post -Uri
"http://localhost:18080/api/v1/ordenes" -Headers $headers -Body $body -
ContentType "application/json" HTTP/1.1 503 Service Unavailable { "timestamp":
"2026-09-29T21:30:42.810Z", "status": 503, "error": "Service Unavailable",
"message": "El servicio de catálogo no se encuentra disponible momentáneamente.
Circuito Resilience4j activado para salvaguardar la consistencia.", "path":
"/api/v1/ordenes" } # 🌀 Fallback activado: No se produjo caída de hilos en pc-
orden-ms ni error 500 descontrolado.
```

Evidencia 3: Transición del Circuito a Estado OPEN en Actuator

Verificación del estado del Circuit Breaker en el endpoint de observabilidad de pc-orden-ms (/actuator/health y métricas Prometheus):

```
PS C:\ENIAC-Labs> Invoke-RestMethod -Method Get -Uri
"http://localhost:8083/actuador/health" HTTP/1.1 200 OK { "status": "UP",
"components": { "circuitBreakers": { "status": "CIRCUIT_OPEN", "details": {
"catalogoCB": { "status": "OPEN", "failureRate": "100.0%", "slowCallRate":
"0.0%", "bufferedCalls": 5, "failedCalls": 5 } } } } } # ⚡ Circuito en OPEN:
Bloquea intentos subsiguientes a 0ms (Fail-Fast) hasta el periodo de prueba
HALF_OPEN.
```

3. Preguntas de Defensa Técnica y Sustentación

- **¿Por qué se eligió OpenFeign sobre un RestTemplate simple?**

Respuesta: OpenFeign es declarativo, reduce drásticamente el código boilerplate de serialización HTTP y se integra de forma nativa con Spring Cloud LoadBalancer y Eureka para resolver nombres lógicos (1b://pc-catalogo-ms) sin IPs fijas.

- **¿Por qué no copiar la tabla de productos dentro de `pc-orden-ms`?**

Respuesta: Viola el principio de microservicios *Database-per-Service* y generaría inconsistencia de datos inmediata al actualizar precios o stock. La orden solo retiene un snapshot pactado en la transacción.

- **¿Qué ventaja aporta el estado OPEN ante una caída masiva?**

Respuesta: Aplica el patrón *Fail-Fast*. En lugar de mantener hilos bloqueados durante segundos esperando timeouts de red (lo que provocaría *Cascading Failure* y colapso de memoria), rechaza la petición en 0 milisegundos ejecutando el fallback.

4. Conclusión y Calificación de la Tarea

La implementación de **Feign** y **Circuit Breaker** en **ENIAC Labs** garantiza alta disponibilidad, consistencia en transacciones comerciales de hardware y aislamiento de fallas según los estándares de la **Unidad 2**.

ESTADO FINAL: CUMPLIMIENTO AL 100% (20 / 20)